

Advanced Terraform on Azure

Lab 8 Landing Zone Consumption

Teaching Points.....	1
Phase 1 – Manual Deployment.....	2
Phase 2 – Create and Consume a Remote State Backend	3
Phase 3 – Import Manually Created Resources	4
Phase 4 – Consume the Landing Zone (Deploy a New Workload)	11
Phase 5 – Migrate the Imported Legacy Resources into the Landing Zone Model	15
Governance Discussion – Why Terraform Surfaces Risk Instead of Hiding It.....	18

Teaching Points

This lab focuses on how workload teams correctly adopt and reconcile with a centrally governed Landing Zone, in line with Azure Cloud Adoption Framework (CAF) principles.

Key concepts reinforced in this lab:

- **Terraform adoption typically begins with existing workloads**
In CAF-aligned environments, workload teams often inherit infrastructure created before platform standardisation. This lab reflects that reality by starting with unmanaged, legacy resources.
- **Terraform state defines workload ownership**
Workload teams are responsible only for resources present in their Terraform state. Import is the mechanism by which workload ownership is recovered without rebuilding infrastructure.
- **Generated configuration supports import, not design authority**
Terraform-generated configuration represents observed state, not architectural intent. Workload teams must review and align imported configuration with Terraform's resource models and platform constraints.
- **Landing Zones are platform-owned and immutable**
In CAF terms, Landing Zones are **platform responsibilities**. Workload teams must consume Landing Zone resources via data sources and must not import, modify, or assume ownership of platform infrastructure.

- **Workload migration follows a replace-then-retire model**
CAF-aligned migration does not mutate platform or legacy designs in place. Workload teams create Landing Zone-aligned replacements first, then explicitly retire non-compliant legacy resources they own.
- **Terraform enables governance by surfacing impact**
Terraform plans make destructive workload changes explicit, enabling platform governance, approval workflows, and risk assessment. Terraform does not conceal or auto-remediate architectural decisions.

By the end of this lab, you should be able to clearly distinguish:

- **Platform-owned resources** (Landing Zone, shared networking)
- **Workload-owned resources** (imported legacy assets and new deployments)
- **Terraform's role** as a reconciliation and governance-enabling tool within a CAF operating model

Phase 1 – Manual Deployment

In this phase you will simulate what often exists **before** an organisation adopts a centralised, Terraform-driven operating model.

A Business Unit has deployed a workload manually, using bespoke networking and security, with no Terraform ownership.

These resources exist in Azure but are unmanaged from Terraform's perspective.

This “legacy, unmanaged” deployment will be imported and reconciled later in the lab.

Seeding the Manual Deployment

The folder **lab8\guided\legacy** contains code that is used purely to seed Azure with resources. Once the resources are created, the Terraform state file produced in this phase will be deliberately destroyed. From that point onward, the resources should be treated as manually created infrastructure. This reflects real-world scenarios where infrastructure exists, but no usable Terraform state is available.

The configuration deploys:

- A legacy Resource Group
- A legacy Virtual Network and Subnet (outside the Landing Zone)
- A legacy Network Interface

All resources are named with a legacy- prefix so they can be easily identified later.

Initialise Terraform and Deploy the Resources

1. From **lab8\guided\legacy**, update **main.tf** with your subscription id.
2. Run ...

```
terraform init
terraform apply --auto-approve
```

3. In the Azure Portal, confirm that:
 - A Resource Group exists for the legacy deployment “**legacy-rg**”
 - The legacy VNet and subnet exist inside that Resource Group
 - The NIC exists and is connected to the legacy subnet

Deliberately Remove Terraform State

To simulate a real-world unmanaged environment, you will now remove Terraform’s memory of the deployment.

4. Delete the entire **lab8\guided\legacy** folder

This concludes the seeding phase.

In the next phase, the Business Unit will establish a remote Terraform backend in preparation for importing these unmanaged resources into Terraform state.

Phase 2 – Create and Consume a Remote State Backend

In this phase, the Business Unit adopts Terraform by first creating a team-safe remote backend for state storage. At this point, no workload resources are being managed yet. The goal of this phase is to prepare Terraform to safely manage infrastructure in the next phases. This reflects a common real-world step: teams first establish secure, shared state management before importing or creating resources.

Create a Business Unit Remote Backend (Bootstrap)

5. Navigate to **artifacts\bu-state-backend-bootstrap**

This folder contains only backend infrastructure code. It is a standalone bootstrap configuration, similar to the one used in Lab 4.

6. Open **terraform.tfvars** and update with ...
 - Your Azure subscription ID
 - A unique suffix for the Storage Account name
7. From the bootstrap folder, run:

```
terraform init
terraform apply --auto-approve
```

This creates the backend infrastructure in Azure.

Configure use the Remote Backend

Now that the backend exists, the Business Units' terraform configuration can be connected to it.

8. Navigate to **lab8\guided** and in **providers.tf**, update `subscription_id` to your Azure subscription ID
9. Review and the backend configuration block, updating the storage account name with your chosen suffix ...

```
backend "azurerm" {
  resource_group_name = "bu-state-backend-rg"
  storage_account_name = "butfstate{suffix}"
  container_name = "tfstate"
  key = "bus-unit-state.tfstate"
}
```

10. Run **terraform init**

11. You should now be able to confirm:

- A remote backend exists in Azure
- Terraform initialises successfully using that backend
- State will be stored centrally in `bus-unit-state.tfstate`
- No workload resources are managed yet

The Business Unit is now ready to import existing Azure resources into Terraform

Phase 3 – Import Manually Created Resources

In this phase you will import the “legacy” resources that already exist in Azure into the Business Unit's remote Terraform state. These resources were “manually created” from Terraform's perspective because the local state was deliberately removed in Phase 1. Terraform currently has **no record** of them.

What you are about to do (pause and think)

What we're doing: Importing existing Azure resources into Terraform state using Azure Resource IDs.

Why: Terraform can only manage what it has in state. Import is how we regain ownership without rebuilding.

Success looks like: terraform state list shows the legacy resources in the BU remote backend.

Collect the Azure Resource IDs for the legacy resources

12. In Azure Portal, locate the legacy resources:

- legacy-vnet
- legacy-subnet
- legacy-nic

13. For each resource; Select **JSON View** and copy the **Resource ID**

14. Record the IDs (e.g., in Notepad). You will need them for import commands.

Tip: The subnet resource ID is easiest to copy from the **Subnet** blade within the VNet, or the VNet JSON view.

Create import blocks (declare import intent in code)

15. In **lab8\guided**, create a new file **imports.tf**

16. Paste in the following, replacing the <...> placeholders with your copied IDs:

```
import {
  to = azurerm_virtual_network.legacy
  id = "<LEGACY_VNET_ID>"
}

import {
  to = azurerm_subnet.legacy
  id = "<LEGACY_SUBNET_ID>"
}

import {
  to = azurerm_network_interface.legacy
  id = "<LEGACY_NIC_ID>"
}
```

Generate starter configuration from the import plan

In this step, Terraform will:

- read the import intent
- inspect the real Azure resources
- generate “best effort” resource blocks into a file

17. At **lab8\guided**, run ...

```
terraform init
```

```
terraform plan --generate-config-out=raw-legacy-generated.tf
```

Expected outcome:

- Terraform produces a plan that includes **import actions**
- Terraform writes a new file: **raw-legacy-generated.tf**
- Error messages are displayed
- The file is still generated successfully — the errors relate to the *generated configuration*, not the import intent

This is Terraform's equivalent of "starter config generation".

It is not guaranteed to be pretty, but it saves you having to guess required blocks.

18. Open and review **raw-legacy-generated.tf** Generated configuration is often verbose. That is normal. This is part of the learning: config generation accelerates import, but does not replace understanding.
19. Save **raw-legacy-generated.tf** as **legacy-generated.tf** and then rename **raw-legacy-generated.tf** to **raw-legacy-generated.old**. Use this as a roll-back if needed.
20. Run **terraform plan** and carefully review the output, there should be three reported issues ...

Error: expected flow_timeout_in_minutes to be in the range (4 - 30), got 0

Error: parsing "": cannot parse an empty string

Error: Value for unconfigurable attribute

21. Open **legacy-generated.tf** and locate the VNet resource block:

```
resource "azurerm_virtual_network" "legacy" { ... }
```

22. Apply the following three edits inside the generated configuration:

Edit A – Remove the invalid flow timeout

Find and delete the line:

```
flow_timeout_in_minutes = 0
```

Edit B – Remove the embedded subnet block

Find and delete the entire subnet = [...] section inside the VNet.

It will look similar to:

```
subnet = [{  
  ...  
  id          = "/subscriptions/.../subnets/legacy-subnet"  
  route_table_id = ""
```

```
security_group = ""  
...  
}]
```

Edit C – Replace the NIC hard-coded subnet ID with a Terraform reference

In resource "azurerm_network_interface" "legacy", locate the NIC's ip_configuration block.

Replace:

```
subnet_id = "/subscriptions/.../subnets/legacy-subnet"
```

with:

```
subnet_id = azurerm_subnet.legacy.id
```

23. Run **terraform plan** again, there should be no reported issues.

Understanding the Generated Configuration Fixes

When Terraform generates starter configuration from imported Azure resources, it produces a **best-effort draft**. That draft may include values and structures that Terraform is **not allowed to configure**, or that do not align with how Terraform models Azure networking.

Refer to **raw-legacy-generated.old** to identify the changes that were required to make the configuration valid and usable ...

Change 1 – Remove an Invalid VNet Flow Timeout

Before (generated): `flow_timeout_in_minutes = 0`

After (fixed): # (Line removed)

Why this matters:

The AzureRM provider only allows values between **4 and 30** for `flow_timeout_in_minutes`. A value of 0 is invalid, even though Azure may behave as if a default is being used.

By removing the argument entirely, Terraform defers to Azure's default behaviour and avoids enforcing an invalid configuration.

Change 2 – Remove the Embedded Subnet Block from the VNet

Before (generated):

```
subnet = [{  
  id      = "/subscriptions/.../subnets/legacy-subnet"  
  route_table_id = ""  
}]
```

```
security_group = ""
service_endpoints = []
}]
```

After (fixed):

Subnet is managed as a separate resource (subnet block removed)

Why this matters:

Terraform does **not** model subnets as inline configuration inside a virtual network. Subnets must be managed using the separate `azurerm_subnet` resource.

The generated block attempted to:

- configure empty string IDs (""), which cannot be parsed
- set `id`, which is a computed attribute and cannot be configured

Removing the embedded block restores Terraform's correct resource model and eliminates multiple validation errors.

Change 3 – Replace Hard-Coded Subnet IDs with Terraform References

Before (generated): `subnet_id = "/subscriptions/.../subnets/legacy-subnet"`

After (fixed): `subnet_id = azurerm_subnet.legacy.id` (line updated)

Why this matters:

Hard-coded Azure resource IDs bypass Terraform's dependency graph and make configurations brittle.

Using a Terraform reference:

- clearly expresses the relationship between resources
- allows Terraform to infer ordering and dependencies
- makes the configuration easier to reason about and evolve

Key Takeaway

Configuration generated during import is **not production-ready**.

It reflects what Terraform *can observe*, not what it *should configure*.

Human review is required to:

- remove invalid or unconfigurable arguments
- align resources with Terraform's modelling patterns
- convert hard-coded IDs into proper references

This review step is a normal and expected part of importing existing infrastructure into Terraform.

Validate State Before Import (Expected Empty State)

Before performing the import, confirm that the Business Unit remote backend currently contains no managed resources.

24. Run **terraform state list**

Expected output:

(no resources)

Why this matters

At this point:

- Azure **does** contain legacy resources
- Terraform **does not** yet own them
- Import intent has been declared, but **state has not been modified**

Terraform does **not** write to state during plan, even when import blocks are present.

This confirms that:

- No accidental state has been created
- Ownership has not yet been established
- The backend is clean and ready to receive imported resources

If resources appear in state at this point, stop and investigate before proceeding.

Apply the Import (Establish Ownership)

Terraform imports resources **only during apply**.

25. Run **terraform apply**

Terraform will display a plan similar to:

Plan: 3 to import, 0 to add, 0 to change, 0 to destroy.

26. When prompted, type: **yes**

Terraform will now:

- Read the real Azure resources
- Write them into the remote backend state
- Associate them with the configuration you have validated

Validate Import Completion

27. After apply completes, run **terraform state list**

Expected output ...

- **azurerm_virtual_network.legacy**
- **azurerm_subnet.legacy**
- **azurerm_network_interface.legacy**

This confirms that:

- The legacy resources are now fully tracked in Terraform state
- State is stored remotely and team-safe
- Terraform ownership has been re-established without rebuilding infrastructure

Confirm No-Op Alignment (Critical Safety Check)

26. Run **terraform plan**

Expected result:

No changes. Infrastructure is up-to-date.

Why this matters

This proves that:

- Configuration matches Azure reality
- Terraform state accurately reflects existing resources
- A future destroy or re-apply would act on the **correct** infrastructure

This step is essential when importing resources that may later be reused, migrated, or rebuilt.

File clean-up

27. Following the successful import, you have the option to sanitize the current file structure. We will do that so that we have consistency moving forward ...

28. Delete the files **import.tf** and **raw-legacy-generated.old**

29. In **legacy-generated.tf**; remove the system generated comments

30. Rename **legacy-generated.tf** to **main.tf**

31. Run **terraform fmt**

Phase Outcome

At the end of this phase, you should be able to state:

- “These resources were manually created in Azure.”
- “Terraform previously had no knowledge of them.”
- “They are now fully managed in a Business Unit remote backend.”
- “Configuration, state, and reality are aligned.”

This completes the import phase.

In the next phase, the Business Unit will deploy **new** workloads directly into the Landing Zone, before returning to **migrating these** imported legacy resources into the centralised platform model.

Phase 4 – Consume the Landing Zone (Deploy a New Workload)

In this phase you will deploy a **new** Business Unit workload into the **DEV Landing Zone**. You will **not** recreate, modify, or import any Landing Zone resources. The Landing Zone is treated as **authoritative and immutable**.

Confirm the target Landing Zone environment

The Landing Zone exists across **DEV/TEST/PROD**. For this lab you must consume **only** the **DEV Landing Zone**.

You will reference Landing Zone resources (Resource Group, VNet, Subnet) from **landing-zone-dev-rg**

In all steps below, use the **DEV** Landing Zone values only.

Review the starting files in lab8\guided

32. Navigate to the lab folder `c:\qatfadvez\lab8\guided` and review the files:

- `providers.tf` (provider + remote backend already configured)
- `variables.tf` (empty)
- `terraform.tfvars` (empty)
- `main.tf` (populated with previous imports)

Generate an SSH Key Pair for the Workload VM

The VM that will be deployed in this phase requires SSH key authentication.

Note: The VM will have **no public IP**, so you will not connect to it using SSH in this lab. The key exists solely to satisfy VM authentication requirements.

33. From PowerShell, run:

```
ssh-keygen -t rsa -b 4096 -f $env:USERPROFILE\.ssh\bu_lab8_key
```

34. When prompted:

Press **Enter** to accept the default file path
Press **Enter** again to leave the passphrase empty

This creates:

bu_lab8_key (private key)
bu_lab8_key.pub (public key)

35. Verify the files exist: **dir \$env:USERPROFILE\.ssh**

Update terraform.tfvars with dev Landing Zone values

36. Open terraform.tfvars and set the Landing Zone lookup values and VM access values.

```
# Landing Zone (dev)
lz_resource_group_name = "landing-zone-dev-rg"
lz_vnet_name = "landing-zone-dev-vnet"
lz_subnet_name = "landing-zone-dev-subnet-app"

# Workload VM access
admin_username = "azureuser"
ssh_public_key_path = "C:/Users/<you>/.ssh/bu_lab8_key.pub"
```

Update variables.tf to support Landing Zone lookups and VM access

37. Open **variables.tf** and add the following variables ...

```
# Landing Zone lookup values (dev only)
variable "lz_resource_group_name" {
  description = "Resource Group name of the dev Landing Zone."
  type        = string
}

variable "lz_vnet_name" {
  description = "Virtual Network name in the dev Landing Zone."
  type        = string
}

variable "lz_subnet_name" {
  description = "Subnet name in the dev Landing Zone."
  type        = string
}

# Workload access
variable "admin_username" {
  description = "Admin username for the workload VM."
  type        = string
}
```

```
variable "ssh_public_key_path" {
  description = "Path to the SSH public key."
  type = string
}
```

Add Landing Zone data sources and workload resources to main.tf

38. Open **main.tf** and add the following configuration.

Look up the dev Landing Zone resources

```
data "azurerm_resource_group" "lz" {
  name = var.lz_resource_group_name
}

data "azurerm_virtual_network" "lz" {
  name = var.lz_vnet_name
  resource_group_name = data.azurerm_resource_group.lz.name
}

data "azurerm_subnet" "lz" {
  name = var.lz_subnet_name
  virtual_network_name = data.azurerm_virtual_network.lz.name
  resource_group_name = data.azurerm_resource_group.lz.name
}
```

Create a NIC in the Landing Zone subnet

```
resource "azurerm_network_interface" "bu_nic" {
  name = "nic-bu-workload"
  location = data.azurerm_resource_group.lz.location
  resource_group_name = data.azurerm_resource_group.lz.name

  ip_configuration {
    name = "ipconfig1"
    subnet_id = data.azurerm_subnet.lz.id
    private_ip_address_allocation = "Dynamic"
  }
}
```

Deploy a small VM using the NIC

```
resource "azurerm_linux_virtual_machine" "bu_vm" {
  name = "vm-bu-workload"
  location = data.azurerm_resource_group.lz.location
  resource_group_name = data.azurerm_resource_group.lz.name
  size = "Standard_B1s"

  admin_username = var.admin_username

  network_interface_ids = [
    azurerm_network_interface.bu_nic.id
  ]
}
```

```

    ]

    admin_ssh_key {
      username   = var.admin_username
      public_key = file(var.ssh_public_key_path)
    }

    os_disk {
      caching           = "ReadWrite"
      storage_account_type = "Standard_LRS"
    }

    source_image_reference {
      publisher = "Canonical"
      offer     = "0001-com-ubuntu-server-jammy"
      sku       = "22_04-lts"
      version   = "latest"
    }
  }
}

```

Plan and apply

39. From c:\qatfadavaz\lab8\guided run:

```
terraform plan
```

Confirm:

- Only the NIC and VM are being created
- No Landing Zone resources are being created or changed

```
terraform apply --auto-approve
```

Verify the workload is consuming the Landing Zone

In the Azure Portal:

- Confirm nic-bu-workload exists
- Confirm vm-bu-workload exists
- Confirm both are deployed into the **dev Landing Zone** Resource Group
- Confirm the NIC is attached to the **dev Landing Zone subnet**

You have deployed a new workload correctly by:

- Referencing Landing Zone resources using **data sources**
- Deploying Business Unit resources into the Landing Zone
- Maintaining clear ownership boundaries

This is the correct operating model for new workloads going forward.

Phase 5 – Migrate the Imported Legacy Resources into the Landing Zone Model

In this phase you will reconcile the Business Unit’s **imported legacy networking** with the centrally managed **DEV Landing Zone**.

Legacy bespoke networking must be retired, and workloads must consume the Landing Zone instead.

What you are about to do (pause and think)

What we’re doing: Migrating BU-owned resources away from bespoke networking, toward the Landing Zone network boundary.

Why: Centralisation only works if legacy patterns are removed, not just prevented going forward.

Success looks like: The legacy VNet/Subnet/NIC are safely removed from Azure **only after** a Landing Zone-aligned replacement exists.

Step 1 — Confirm the legacy resources are still present and BU-owned

40. From `c:\qatfadavaz\lab8\guided`, run **terraform state list**

Expected outcome: you can see your imported legacy resources in state (from Phase 3), for example:

- `azurerm_virtual_network.legacy`
- `azurerm_subnet.legacy`
- `azurerm_network_interface.legacy`

This confirms the BU currently *owns* the legacy resources in Terraform and therefore has the ability to retire them safely.

Step 2 — Plan the migration approach (replace, don’t mutate)

The Landing Zone is immutable. You must **not** modify it. Because of that, the typical migration pattern is:

1. Create a **Landing Zone-aligned replacement** (new NIC in LZ subnet)
2. Retire the **legacy networking** (old NIC, subnet, VNet)

This is the realistic approach in enterprises: you don’t “move the platform”, you **move off** the legacy design.

Step 3 — Add a “replacement NIC” into the Landing Zone

41. In `lab8\guided\main.tf`, add a second NIC resource **alongside** the existing resources:

```
# Replacement NIC that consumes the Landing Zone subnet
resource "azurerm_network_interface" "legacy_replacement" {
  name = "legacy-nic-replacement"
  location = data.azurerm_resource_group.lz.location
  resource_group_name = data.azurerm_resource_group.lz.name

  ip_configuration {
    name = "ipconfig1"
    subnet_id = data.azurerm_subnet.lz.id
    private_ip_address_allocation = "Dynamic"
  }
}
```

Notes:

- This NIC is deliberately created in the **Landing Zone Resource Group**, using the existing data sources from Phase 4.
- We are not “editing legacy” yet — we are creating the safe destination first.

Step 4 — Run a plan and confirm we are only adding the replacement NIC

41. Run **terraform plan**

Confirm the plan shows:

1 to add (the replacement NIC) **0 to change** **0 to destroy**

If Terraform shows changes to Landing Zone resources, stop. You should only be consuming the LZ via data sources.

Step 5 — Apply the creation of the replacement NIC

42. Run: **terraform apply --auto-approve**

43. In the Azure Portal, verify:

- legacy-nic-replacement exists
- It is deployed into the **Landing Zone DEV Resource Group**
- It is attached to the **Landing Zone DEV subnet**

At this point you have proven a safe migration principle: “Create the compliant replacement **first**.”

Step 6 — Retire the legacy networking (decommission)

Now that a Landing Zone-aligned NIC exists, the legacy networking can be removed.

44. In main.tf, remove the legacy resources you imported earlier:

- azurerm_network_interface.legacy

- `azurerm_subnet.legacy`
- `azurerm_virtual_network.legacy`

45. Run: **terraform plan**

Expected outcome:

- **0 to add, 0 to change, 3 to destroy** (legacy NIC, legacy subnet, legacy VNet)

Terraform doesn't hide destructive change — it surfaces it clearly so it can be governed.

Step 7 — Apply the retirement

46. Run: **terraform apply --parallelism=1 --auto-approve**

47. Confirm the destroys complete successfully.

48. Switch to the Portal and manually delete the now empty **legacy-rg** resource group.

Step 8 — Validate the end state

48. Run: **terraform state list**

Expected outcome:

- No legacy VNet/Subnet/NIC remain
- The Landing Zone replacement NIC remains
- Phase 4 resources remain (your native LZ workload NIC + VM)

49. In the Azure Portal, verify:

- The legacy resource group no longer contains the VNet/Subnet/NIC
- The Landing Zone DEV resource group still contains:
 - the Phase 4 workload resources
 - the replacement NIC created in this phase

Phase 5 Outcome

You should now be able to state:

- “We imported unmanaged resources into Terraform state (recovered ownership).”
- “We created new workloads correctly in the Landing Zone (future behaviour).”
- “We retired legacy bespoke networking only after a compliant replacement existed.”
- “Terraform made the destructive nature of migration explicit, enabling governance and decision-making.”

Governance Discussion – Why Terraform Surfaces Risk Instead of Hiding It

This phase intentionally ended with **destructive actions** being clearly presented in the Terraform plan. That was not an accident. In centrally governed environments, the most dangerous changes are not the ones that Terraform blocks — they are the ones that happen silently. Terraform's value here is not that it prevents destruction, but that it **forces organisations to confront it explicitly**.

What Terraform Made Visible

During this phase, Terraform clearly showed:

- Which resources would be destroyed
- That those resources were legacy, bespoke, and no longer compliant
- That destruction would only occur **after** a Landing Zone-aligned replacement existed

This transparency enables informed decision-making rather than accidental change.

Why This Matters in Regulated Environments

In real organisations, this plan output would typically trigger:

- A formal approval workflow
- A change record or Change Advisory Board review
- Risk assessment and scheduling (for example, maintenance windows)

Terraform does not replace governance — it **supports it** by providing accurate, deterministic information about impact and blast radius.

Platform vs Workload Responsibility Revisited

At no point in this phase did the Business Unit:

- Modify Landing Zone networking
- Import Landing Zone resources
- Assume ownership of platform infrastructure

This aligns directly with Azure CAF guidance:

- Platform teams own shared infrastructure
- Workload teams consume, migrate, and adapt

The destructive actions performed here were strictly **within Business Unit ownership boundaries**, which is exactly where Terraform state should be authoritative.

Why Terraform Does Not “Auto-Fix” Legacy Design

A common misconception is that Terraform should automatically migrate or refactor infrastructure into compliance. Terraform deliberately does **not** do this.

Instead, it:

- Surfaces drift
- Highlights incompatibilities
- Makes destructive changes explicit
- Requires human approval for irreversible actions

This ensures architectural decisions remain **human-led**, not tool-driven.

Key Takeaway

Centralisation does not erase history.

Terraform provides a mechanism to:

- Recover ownership
- Understand legacy decisions
- Introduce compliant patterns
- Retire outdated designs safely and deliberately

Progress happens through **reconciliation**, not denial.

This is why modern Terraform usage in enterprises focuses as much on governance, visibility, and migration as it does on creation.

Copyright

© 2026 QA Michael Coulling-Green. All rights reserved. These lab materials are provided as part of an authorised training course. Course attendees may reuse these materials for their own personal learning and reference outside of the training session. Unauthorised copying, redistribution, publication, or use of these materials to deliver training is prohibited without prior written permission from the author.

© QA 2026